

Poglavje 8 **P4** → **V4** Odjemalec/strežnik in Node.js



P4 → **V4**

V pripravi na vaje **V4** bo predstavljen kratek uvod v [Node.js](#):

- **enostavni spletni strežnik** (Hello World; testiranje in ponovni zagon; spremenljivke na strežniku),
- **napredni spletni strežnik** (strežnik statičnih spletnih strani; uporaba dodatnih knjižnic; obvladovanje zahtev po virih, ki ne obstajajo; predpomnjenje; dinamične strani; pridobivanje podatkov uporabnika; shranjevanje v datoteko).

Celotna izvorna koda primera v nadaljevanju je na voljo v GitHub repozitoriju [P4-V4](#). Najprej na GitHub izdelajte kopijo projekta (*angl. Fork*), nato pa si lokalno okolje nastavite z naslednjima ukazoma:

```
git clone git@github.com:{VAŠE-GITHUB-UPORABNIŠKO-IME}/P4-V4.git
cd P4-V4
```

ali

```
git clone https://github.com/{VAŠE-GITHUB-UPORABNIŠKO-IME}/P4-V4.git
cd P4-V4
```

Pri delu bomo uporabljali razvojno okolje [Node.js](#), ki ga morate namestiti na svoj računalnik. Node.js je odprtokodno ogrodje, ki omogoča izvajanje kode v programskem jeziku JavaScript na strani strežnika in deluje na vseh pogosto uporabljenih platformah (Windows, Linux in macOS).

8.1 Razvoj enostavnega spletnega strežnika

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Enostavni strežnik**.

8.1.1 Hello World

Kreiramo najbolj enostavno spletno aplikacijo, ki izpiše zgolj pozdrav. Uporabimo vgrajeno knjižnico `http` in metodi `http.createServer` za kreiranje objekta strežnika ter `server.listen` za zagon strežnika na izbranih vratih (angl. *W port*).

streznik_enostavni.mjs

```
/**
 * Če so vrata strežnika opredeljena v okoljski spremenljivki PORT,
 * uporabimo to vrednost, sicer nastavimo vrata na vrednost 8080.
 */
const port = process.env.PORT || 8080;

import { createServer } from "http";

createServer(function (zahteva, odgovor) {
  odgovor.writeHead(200, { "Content-Type": "text/plain" });
  odgovor.end("Pozdravljeni vsi ljubitelji predmeta OIS!\n");
})
.listen(port);

console.log("Strežnik je pognan!");
```

200 - vse ok

kako deluje strežnik

izpiše na konzolo (terminal)

Tukaj se zopet (prvič smo ga omenili na prejšnjih pripravah na vaje **P3** → **V3** v okviru poglavja *Dogodki in povratni klici*) srečamo s funkcionalnostjo **povratnega klica** (angl. `callback`), ki je zelo pogost vzorec pri JavaScript asinhronih operacijah.

Funkcija **povratnega klica** je funkcija (npr. `funkcijaX`), ki jo posredujemo kot parameter pri klicu druge funkcije (npr. `funkcijaY`) in ta funkcija povratnega klica (`funkcijaX`) se izvede znotraj druge funkcije (`funkcijaY`). Več o tej funkcionalnosti si lahko preberete v prispevku [Understand JavaScript Callback Functions and Use Them](#).

8.1.1.1 Zagon strežnika in pregled delovanja

Strežniško aplikacijo, ki smo jo ravnokar razvili, lahko v lokalnem razvojnem okolju poženemo z uporabo ukaza `node streznik_enostavni.mjs`, ki ga vnesemo v terminal.

8.1.1.2 Spremembe in ponovni zagon

Če spremenimo izpis (glej spodaj), nam strežnika ni potrebno ponovno pognati, v brskalniku samo osvežimo stran in imamo na voljo novo vsebino!

streznik_enostavni.mjs

```
createServer(function (zahteva, odgovor) {
  odgovor.writeHead(200, { "Content-Type": "text/plain" });
  odgovor.end(
    "Pozdravljeni vsi ljubitelji predmeta OIS in ostali slepi potniki!\n"
  );
}).listen(port);
```

Če so spremembe na strani strežnika večje, potem moramo strežnik ponovno pognati (v terminalu trenutno aktivni strežnik ustavimo (npr. s kombinacijo tipk `CTRL / CMD + C` in ga ponovno poženemo s prej omenjenim ukazom `node enostavni_streznik.mjs`).

8.2 Spremenljivke na strežniku

Dodajmo enostaven **števec uporabnikov**, ki hrani število uporabnikov v **globalni spremenljivki na strežniku**.

streznik_enostavni.mjs

```
import { createServer } from "http";
let steviloUporabnikov = 0;
createServer(function (zahteva, odgovor) {
  steviloUporabnikov++;
  odgovor.writeHead(200, { "Content-Type": "text/plain" });
  odgovor.end(
    "Pozdravljeni vsi ljubitelji predmeta OIS in ostali slepi potniki!\n"
    "Pozdravljen " + steviloUporabnikov + ". ljubitelj predmeta OIS!\n"
  );
}).listen(port);
```

8.3 Strežnik statičnih spletnih strani

Kreirali bomo novo datoteko `streznik_napredni.mjs`, kjer bo implementiran strežnik, ki nam bo stregel statične spletne strani, ki smo jih razvili prejšnji teden na pripravah na vaje **P3** → **V3** [JavaScript na strani odjemalca](#). Datoteke bomo organizirali v naslednjo strukturo:

Datoteka/mapa	Opis
 node_modules	Knjižnice, potrebne za projekt
 formidable	
 mime-types	
 public	Statične datoteke, ki jih strežnik streže odjemalcu
 anakonda.jpg	
 index.html	
 koda.js	
 stil.css	
 package-lock.json	Opređeljene konkretne verzije odvisnosti od knjižnic
 package.json	Opređeljene odvisnosti od knjižnic
 streznik_enostavni.mjs	Logika delovanja enostavnega strežnika
 streznik_napredni.mjs	Logika naprednega strežnika

8.3.1 Knjižnice, ki jih bo naš projekt uporabljal

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Opređeljene odvisnosti od knjižnic**.

V datoteki `package.json` opredelimo odvisnosti projekta od knjižnic, in sicer  `mime-types` in  `formidable`.
če določena knjižnica še ni opredeljena, jo naredimo

`package.json`

```
{
  "name": "napredni-streznik",
  "version": "0.0.1",
  "description": "Strežnik, ki streže statične in dinamične spletne strani",
  "dependencies": {
    "mime-types": "*",
    "formidable": "*"
  },
  "private": true
}
```

tip odgovora, ki ga strežnik vrne odjemalcu (če prebere sliko, mora sporočit odjemalcu, da je to slika)

Z ukazom `npm install` node packet manager zahtevamo namestitev teh knjižnic v mapo `node_modules`, da jih lahko naš strežnik tudi uporablja. Poleg omenjene mape, se ustvari tudi nova datoteka `package-lock.json`, kjer so zapisane odvisnosti od konkretnih verzij knjižnic, ki jih bomo uporabljali. Opazimo lahko, da so poleg dveh zahtevanih knjižnic nameščene tudi še ostale odvisne knjižnice.

Konzola

`node_modules` ni smiselno dajat na gitHub, zato jo dodamo v datoteko `.gitignore`

```
$ npm install
added 8 packages, and audited 9 packages in 1s
1 package is looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

Vsebina mape `node_modules` predstavlja knjižnice, ki jih ne shranjujemo v sistem git, saj jih lahko v vsakem trenutku namestimo z ukazom `npm install`. Zato mape `node_modules` ne bomo vključili v spremljanje spremembe pri kreiranju uveljavitve tako, da v datoteko `.gitignore` dodamo to omejitev.

```
.gitignore
```

```
.idea
.vs
.DS_Store
node_modules
```

Poleg že omenjene vgrajene knjižnice `http` in dodatnih knjižnic `mime-types` ter `formidable` bomo uporabili tudi vgrajeni knjižnici  `fs` za dostop do datotečnega sistema in  `path` za pridobitev ustreznih poti do datotek.

8.3.2 Uvoz statičnih datotek

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Statične spletne strani iz prejšnjih priprav na vaje**.

Najprej kreiramo mapo `public`, v katero s pristopom povleci in spusti vključimo vse statične datoteke iz priprav na vaje prejšnjega tedna.

8.3.3 Začnemo s pisanjem kode na strežniku

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Strežnik statičnih spletnih strani**.

Kreiramo datoteko `streznik_napredni.mjs` in opredelimo osnovne odvisnosti.

```
streznik_napredni.mjs
```

```
/**
 * Če so vrata strežnika opredeljena v okoljski spremenljivki PORT,
 * uporabimo to vrednost, sicer nastavimo vrata na vrednost 8080.
 */
const port = process.env.PORT || 8080;

// Opredelimo knjižnice, ki jih potrebujemo
import { lookup } from "mime-types";
import { createServer } from "http";
import { access, readFile } from "fs"; // file system
import { basename } from "path";     // za preslikovanje relativnih in absolutnih poti
```

8.3.4 Pripravimo objekt strežnika in ga osnovno nastavimo

streznik_napredni.mjs

```
// Kreiramo objekt strežnika in mu nastavimo osnovne parametre
const streznik = createServer((zahteva, odgovor) => {
  let potDoDatoteke = false;
  if (zahteva.url == "/") {
    potDoDatoteke = "./public/index.html";
  } else {
    potDoDatoteke = "./public" + zahteva.url;
  }
  posredujStaticnoVsebino(odgovor, potDoDatoteke);
});
```

namesto function(zahteva, odgovor) {

8.3.5 Strežnik poženemo

streznik_napredni.mjs

```
// Na tej točki strežnik poženemo
streznik.listen(port, () => {
  console.log("Strežnik pognan!");
});
```

8.3.6 Pripravimo še manjkajočo metodo za strežbo statične vsebine

Uporabili bomo metodo `fs.access`, ki preveri, ali datoteka na datotečnem sistemu obstaja, in jo bomo v primeru obstoja prebrali z metodo `fs.readFile`.

streznik_napredni.mjs

```
// Metoda za posredovanje statične vsebine (datoteke iz mape public)
function posredujStaticnoVsebino(odgovor, potDoDatoteke) {
  access(potDoDatoteke, (napaka) => { ali ta datoteka sploh obstaja
    if (!napaka) { access v primeru, da ne najde datoteke, naredi objekt napake, sicer ne
      readFile(potDoDatoteke, (napaka, datotekaVsebina) => {
        if (napaka) {
          posredujNapako(odgovor, 500); če pri readFile pride do napake, vrne napako, sicer vrne datoteko
        } else {
          posredujDatoteko(odgovor, potDoDatoteke, datotekaVsebina);
        }
      });
    } else {
      posredujNapako(odgovor, 404); če že pri access pride do napake vrne napako drugega tipa
    }
  });
}
```

8.3.7 Pripravimo metodo za obvladovanje napak

Če je odgovornost za napako na strani odjemalca, vrnemo `napako 404`, saj gre za zahtevo po neobstoječem viru. V primeru napake strežnika pa uporabniku vrnemo `napako 500`.

streznik_napredni.mjs

```
// Obvladovanje napak če se napaka začne s 4, je kriv odjemalec, če se začne s 5, je kriv strežnik
function posredujNapako(odgovor, tip) {
  odgovor.writeHead(tip, { "Content-Type": "text/plain" });
  if (tip == 404) {
    odgovor.end("Napaka 404: Vira ni mogoče najti.");
  } else if (tip == 500) {
    odgovor.end("Napaka 500: Prišlo je do napake strežnika.");
  } else {
    odgovor.end("Napaka " + tip + ": Neka druga napaka");
  }
}
```

8.3.8 Pripravimo tudi metodo, ki vrne datoteko odjemalcu

streznik_napredni.mjs

```
// Metoda, ki vrne datoteko in nastavi tip datoteke,  
// da jo brskalnik zna ustrezno prikazati  
function posredujeDatoteko(odgovor, datotekaPot, datotekaVsebina) {  
  odgovor.writeHead(200, {  
    "Content-Type": lookup(basename(datotekaPot)),  
    // slika, css, ...           celotna absolutna pot  
  });  
  odgovor.end(datotekaVsebina); // vrne vsebino datoteke  
}
```

8.3.9 Preverimo delovanje strežbe statičnih strani

Preverimo delovanje trenutne verzije aplikacije:

- pokažemo začetno stran,
- zahtevamo sliko `anakonda.jpg`,
- zahtevamo CSS stil `stil.css`,
- poskušamo dostopati do neobstoječe strani.

8.4 Strežniku statičnih strani dodamo predpomnjenje

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Dodano predpomnjenje**.

Najprej v datoteki `streznik_napredni.mjs` opredelimo objekt `predpomnilnik`, kamor bomo shranjevali statične datoteke.

streznik_napredni.mjs

```
...  
import { basename } from "path";  
let predpomnilnik = {};  
// Kreiramo objekt strežnika in mu nastavimo osnovne parametre  
...
```

Nato posodobimo metodo `posredujeStaticnoVsebino` za posredovanje statične vsebine datotek.

streznik_napredni.mjs

```
// Metoda za posredovanje statične vsebine (datoteke iz mape public)
function posredujeStaticnoVsebino(odgovor, potDoDatoteke) {
  // Preverjanje, ali je datoteka v predpomnilniku
  if (predpomnilnik[potDoDatoteke]) {
    posredujeDatoteko(odgovor, potDoDatoteke, predpomnilnik[potDoDatoteke]);
  } else {
    access(potDoDatoteke, (napaka) => {
      if (!napaka) {
        readFile(potDoDatoteke, (napaka, datotekaVsebina) => {
          if (napaka) {
            posredujeNapako(odgovor, 500);
          } else {
            // Shranjevanje vsebine nove datoteke v predpomnilnik
            predpomnilnik[potDoDatoteke] = datotekaVsebina;
            posredujeDatoteko(odgovor, potDoDatoteke, datotekaVsebina);
          }
        });
      } else {
        posredujeNapako(odgovor, 404);
      }
    });
  }
}
}
```

8.5 Strežniku dodamo dinamične strani

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Dinamična stran s pisanjem v datoteko**.

V datoteko želimo shraniti vse komentarje, ki jih uporabniki pustijo na naši strani.

8.5.1 Popravimo nastavitve strežnika

Strežniku moramo sporočiti, kdaj naj se odzove na poslane podatke.

```
streznik_napredni.mjs
```

```
// Kreiramo objekt strežnika in mu nastavimo osnovne parametre
const streznik = createServer((zahteva, odgovor) => {
  let potDoDatoteke = false;
  if (zahteva.url == "/") {
    potDoDatoteke = "./public/index.html";
  } else if (zahteva.url == "/posredujeSporocilo") {
    obdelajSporocilo(zahteva, odgovor);
  } else {
    potDoDatoteke = "./public" + zahteva.url;
  }
  // Statično vsebino postrežemo le takrat, ko gre za zahtevo takšne vrste
  if (potDoDatoteke) posredujeStaticnoVsebino(odgovor, potDoDatoteke);
});
```

8.5.2 Implementiramo metodo za obdelavo sporočila

Uporabili bomo knjižnico  `formidable`, ki nam omogoča enostavno razčlenjevanje posredovanih podatkov z metodo `parse` nad objektom  `formidable.IncomingForm` preko spletnega obrazca. V primeru napake na strani strežnika o tem obvestimo tudi uporabnika. Če so podatki pravilno posredovani, jih shranimo v datoteko `sporocila.txt` in se zahvalimo uporabniku.

```
streznik_napredni.mjs
```

```

// Opredelimo knjižnice, ki jih potrebujemo
import { lookup } from "mime-types";
import { createServer } from "http";
import { access, readFile, appendFile } from "fs";
import { basename } from "path";
import { IncomingForm } from "formidable";

// Metoda, ki obdela sporočilo na strežniku
function obdelajSporocilo(zahteva, odgovor) {
  let obrazec = new IncomingForm();
  obrazec.parse(zahteva, (napaka, vrednosti) => {
    if (napaka) {
      posredujNapako(odgovor, 404);
    } else {
      let sporociloUporabnika =
        new Date().toJSON() +
        "|" +
        vrednosti.oseba +
        "|" +
        vrednosti.komentar +
        "|" +
        vrednosti.strah +
        "\n";
      appendFile("./sporocila.txt", sporociloUporabnika, (napaka) => {
        if (napaka) {
          posredujNapako(odgovor, 500);
        } else {
          posredujZahvaloZaPosredovanKomentar(odgovor);
        }
      });
    }
  });
}

```

8.5.3 Implementiramo zahvalo uporabniku

streznik_napredni.mjs

```
// Metoda, ki se zahvali uporabniku
function posredujeZahvaloZaPosredovanKomentar(odgovor) {
  odgovor.writeHead(200, { "Content-Type": "text/plain" });
  odgovor.end("Hvala za posredovane komentarje!");
}
```

8.6 Testiranje in dodatno gradivo

Razvito aplikacijo lahko preverimo tako, da jo damo na voljo še ostalim uporabnikom. Komentarji bi se morali hraniti v datoteki `sporocila.txt`.

Za lažji uvod v okolje **Node.js** so na voljo različni viri, ki jih lahko uporabite pri spoznavanju tega okolja:

- [Introduction to Node.js](#),
- [NodeSchool](#) interaktivne lekcije,
- [W3Schools Node.js tutorial](#),
- [Node.js Example](#).